

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 5- Divide & Conquer

References:

- ❖ *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*

Divide & Conquer

Introduction

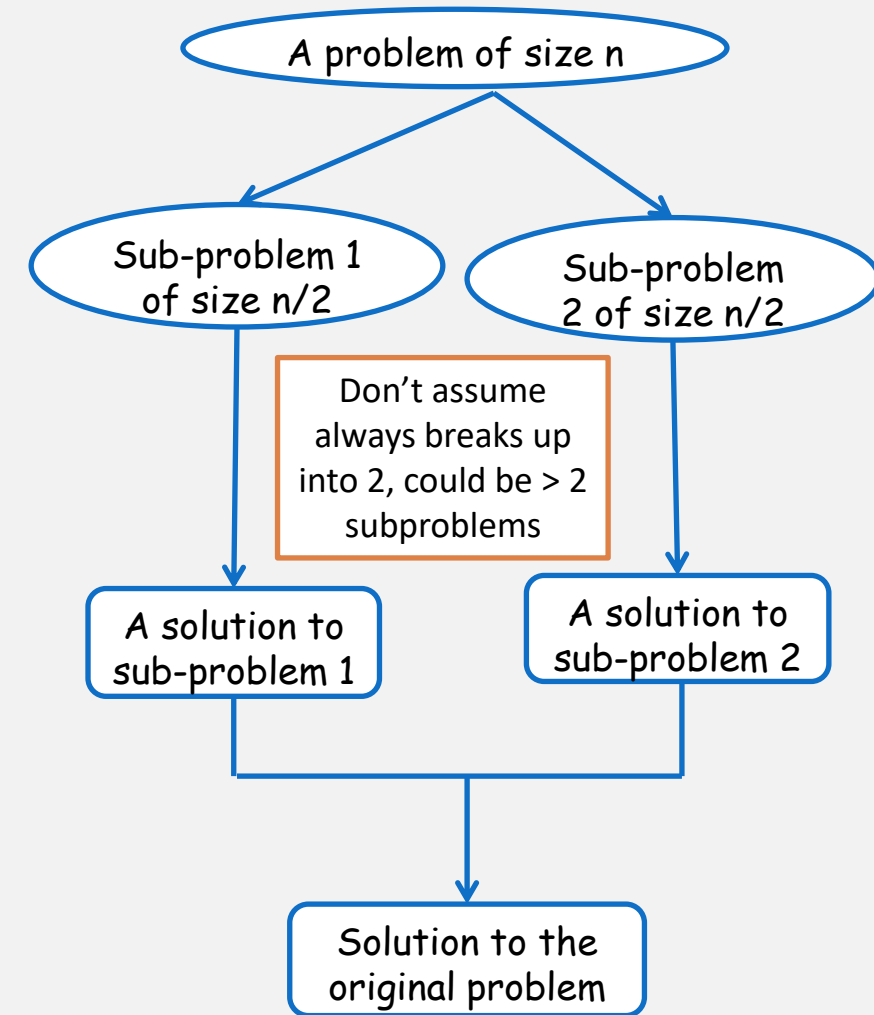
The ***divide-and-conquer*** technique work according to the following general plan:

1. Divide instance of problem into two or more smaller sub-problems.
2. Solve the resulting sub-problems (i.e., smaller instances) recursively.
3. Combine all solutions of the sub-problems to obtain a solution to the original problem.

Divide and Conquer is not the best technique always

Example $a_0 + \dots + a_{n-1} = (a_0 + \dots + a_{\lfloor n/2 \rfloor - 1}) + (a_{\lfloor n/2 \rfloor} + \dots + a_{n-1})$

Number of additions is same as in brute force, needs stack for recursion !!



Divide & Conquer

Examples

- Sorting: mergesort and quicksort
- Binary tree traversals
- Multiplication of large integers
- Matrix multiplication: Strassen's algorithm
- Closest-pair and convex-hull algorithms
- Binary search: decrease-by-half (or degenerate divide&conq.)

Divide & Conquer

General Divide-and-Conquer Recurrence

$$T(n) = a T(n/b) + f(n)$$

a of the b instances needs to be solved

time spent in dividing an instance of size n into sub-problems of size n/b and combining their solution

an instance of size n can be divided into b instances of size n/b

$$T(n) = aT(n/b) + f(n) \quad \text{where } f(n) \in \Theta(n^d), \quad d \geq 0$$

Master Theorem: If $a < b^d$, $T(n) \in \Theta(n^d)$

If $a = b^d$, $T(n) \in \Theta(n^d \log n)$

If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

Note: The same results hold with O instead of Θ .

Divide & Conquer

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$
If $f(n) \in \Theta(n^d)$ where $d \geq 0$, then

$\Rightarrow T(n) \in$

$\left\{ \begin{array}{ll} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \lg n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{array} \right.$

$T(n)$	a	b	$f(n)$	d	Case	Result
$3 T(n/2) + n$	3	2	$\Theta(n^1)$	1	$a=3 > b^d=2^1$	$\Theta(n^{\log_2 3}) = \Theta(n^{1.5850})$
$3 T(n/2) + n^2$	3	2	$\Theta(n^2)$	2	$a=3 < b^d=2^2$	$\Theta(n^2)$
$4 T(n/2) + n^2$	4	2	$\Theta(n^2)$	2	$a=4 = b^d=2^2$	$\Theta(n^2 \log n)$
$0.5 T(n/2) + 1/n$					Master th ^m doesn't apply, $a < 1$, $d < 0$	
$2 T(n/2) + n/\lg n$					Master th ^m doesn't apply $f(n)$ not polynomial	
$64 T(n/8) - n^2 \lg n$					$f(n)$ is not positive, doesn't apply	
$2^n T(n/8) + n$					a is not constant, doesn't apply	

Divide & Conquer

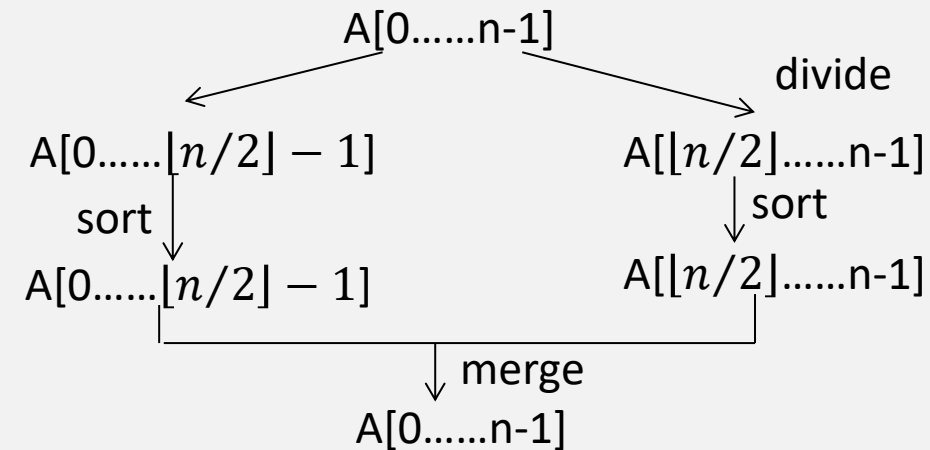
Sorting: Mergesort

- Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
 - Repeat the following until no elements remain in one of the arrays:
 - compare the first elements in the remaining unprocessed portions of the arrays
 - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
 - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Divide & Conquer

Sorting: Mergesort

- Divide the list into two smaller lists of about equal sizes
- Sort each smaller list *recursively*
- Merge the two sorted lists to get one sorted list



ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

copy $A[0..n/2 - 1]$ to $B[0..n/2 - 1]$

copy $A[n/2..n - 1]$ to $C[0..n/2 - 1]$

Mergesort($B[0..n/2 - 1]$)

Mergesort($C[0..n/2 - 1]$)

Merge(B, C, A)

- If the input list is a linked list, dividing takes $O(N)$ time
- If the input list is an array $A[0..N-1]$: dividing takes $O(1)$ time
- we can represent a sublist by two integers *left* and *right*: to divide $A[\text{left}..\text{Right}]$, we compute $\text{center} = (\text{left} + \text{right}) / 2$ and obtain $A[\text{left}..\text{center}]$ and $A[\text{center} + 1..\text{Right}]$

Divide & Conquer

Sorting: Mergesort

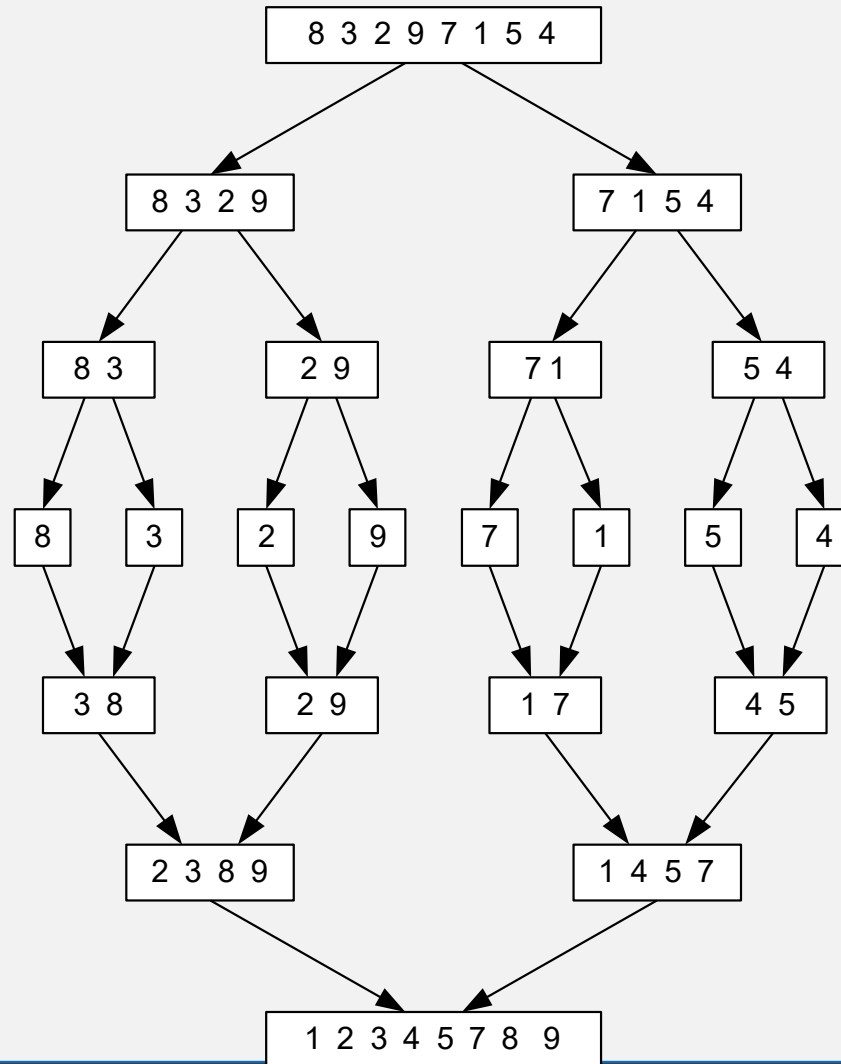
ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)
 //Merges two sorted arrays into one sorted array
 //Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
 //Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
 $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$
while $i < p$ and $j < q$ **do**
 if $B[i] \leq C[j]$
 $A[k] \leftarrow B[i]; i \leftarrow i + 1$
 else $A[k] \leftarrow C[j]; j \leftarrow j + 1$
 $k \leftarrow k + 1$
 if $i = p$
 copy $C[j..q-1]$ to $A[k..p+q-1]$
 else copy $B[i..p-1]$ to $A[k..p+q-1]$

Takes $\Theta(p+q)$, where p and q represents respectively the size of array B and array C .

1	13	24	26	2	15	27	38	1									
1	13	24	26	2	15	27	38	1	2								
1	13	24	26	2	15	27	38	1	2	13							
1	13	24	26	2	15	27	38	1	2	13	15						
1	13	24	26	2	15	27	38	1	2	13	15	24					
1	13	24	26	2	15	27	38	1	2	13	15	24	26				
1	13	24	26	2	15	27	38	1	2	13	15	24	26	27			
1	13	24	26	2	15	27	38	1	2	13	15	24	26	27	38		

Divide & Conquer

Mergesort Example



Divide & Conquer

Sorting: Analysis of Mergesort

Let $T(N)$ denote the worst-case running time of mergesort to sort N numbers.

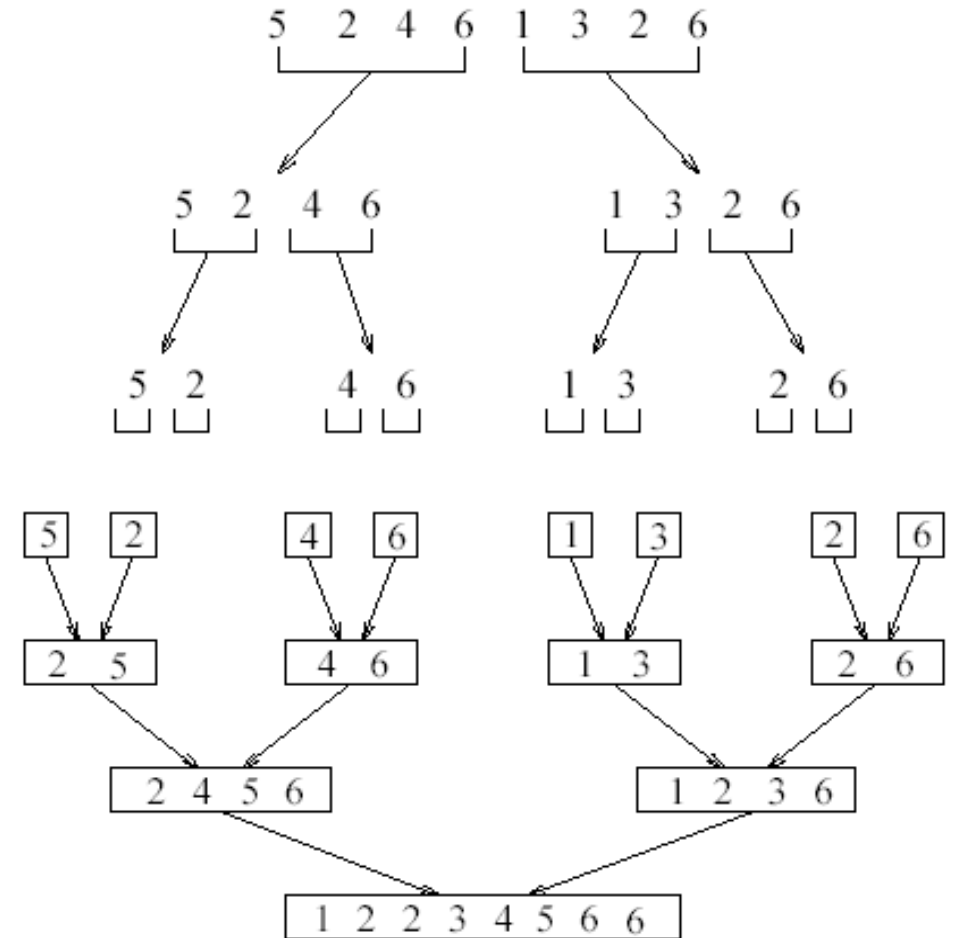
Assume that N is a power of 2.

- Divide step: $O(1)$ time
- Conquer step: $2 T(N/2)$ time
- Combine step: $O(N)$ time

Recurrence equation:

$$\begin{cases} T(1) = 1 \\ T(N) = 2T(N/2) + N \end{cases}$$

according to the Master Theorem, $C_{worst}(n) \in (n \log n)$



Divide & Conquer

Sorting: Analysis of Mergesort

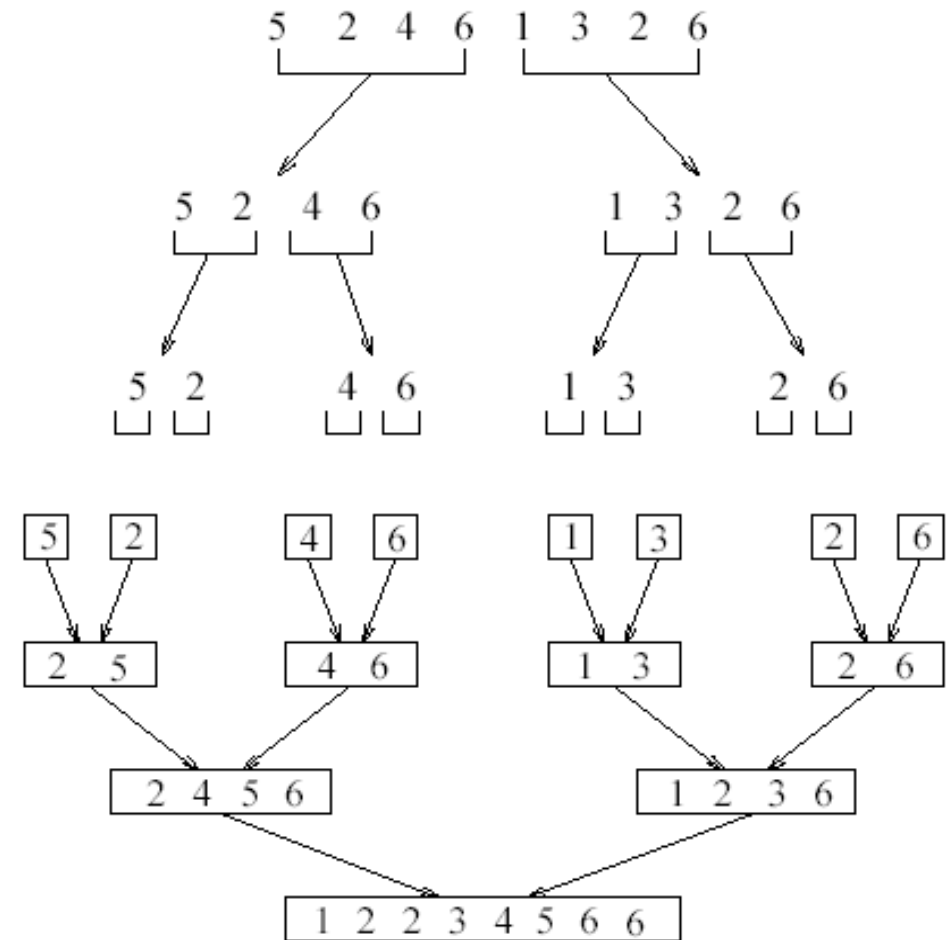
All cases have same efficiency: $\Theta(n \log n)$

Number of comparisons in the worst case is close to theoretical minimum for comparison-based sorting:

$$\lceil \log_2 n! \rceil \approx n \log_2 n - 1.44n$$

Space requirement: $\Theta(n)$ (not in-place)

Can be implemented without recursion (bottom-up)



Divide & Conquer

Sorting: Quicksort

- Divide step:
 - Pick an element (pivot) v in S
 - Partition $S - \{v\}$ into two disjoint groups
$$S1 = \{x \in S - \{v\} \mid x < v\}$$
$$S2 = \{x \in S - \{v\} \mid x \geq v\}$$
- Conquer step: recursively sort $S1$ and $S2$
- Combine step: combine the sorted $S1$, followed by v , followed by the sorted $S2$

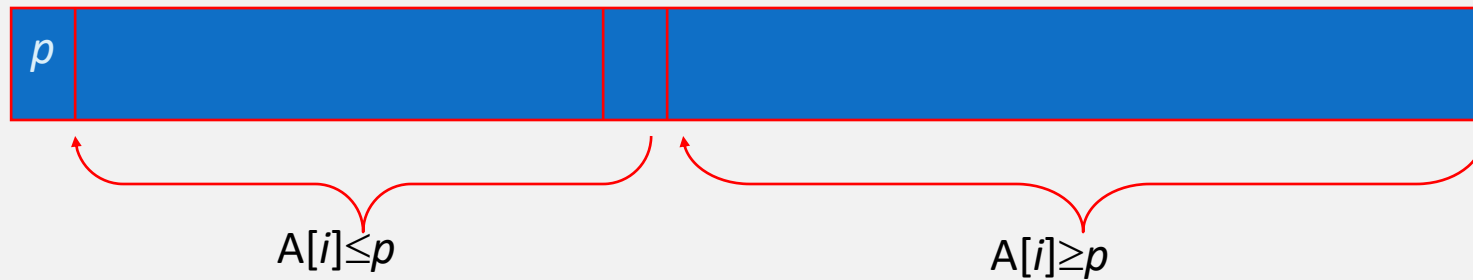


$$\underbrace{A[0] \dots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \dots A[n-1]}_{\text{all are } \geq A[s]}$$

Divide & Conquer

Sorting: Quicksort

- Select a *pivot* (partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining $n-s$ positions are larger than or equal to the pivot (see next slide for an algorithm)



- Exchange the pivot with the last element in the first (i.e., $\leq$) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

Divide & Conquer

Sorting: Quicksort

ALGORITHM *Quicksort*($A[l..r]$)

//Sorts a subarray by quicksort

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right
// indices l and r

//Output: Subarray $A[l..r]$ sorted in non-decreasing order

if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$ // s is a split position

Quicksort($A[l..s - 1]$)

Quicksort($A[s + 1..r]$)

Pick a pivot, many different ways:

1. Always pick first element as pivot.
2. Always pick last element as pivot
3. Pick a random element as pivot.
4. Pick median as pivot.



Choosing a partitioning algorithm

1. Lomuto Partition
2. Hoare Partition

Divide & Conquer

Sorting: Quicksort (Hoare's Partitioning Algorithm)

ALGORITHM *HoarePartition*($A[l..r]$)

//Partitions a subarray by Hoare's algorithm, using the first element

// as a pivot

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right

// indices l and r ($l < r$)

//Output: Partition of $A[l..r]$, with the split position returned as

// this function's value

$p \leftarrow A[l]$

$i \leftarrow l; j \leftarrow r + 1$

repeat

repeat $i \leftarrow i + 1$ **until** $A[i] \geq p$

repeat $j \leftarrow j - 1$ **until** $A[j] \leq p$

swap($A[i]$, $A[j]$)

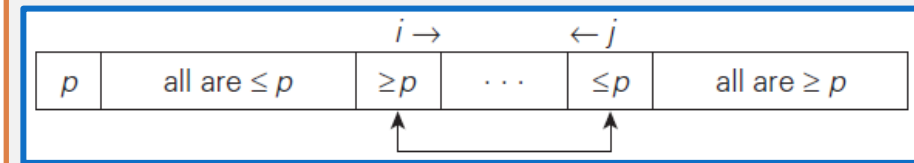
until $i \geq j$

swap($A[i]$, $A[j]$) //undo last swap when $i \geq j$

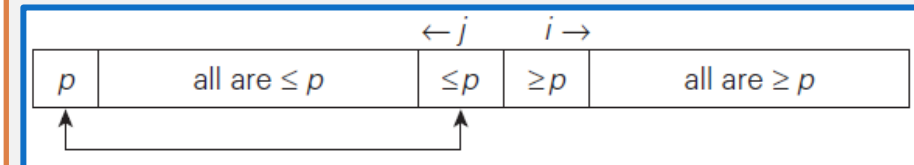
swap($A[l]$, $A[j]$)

return j

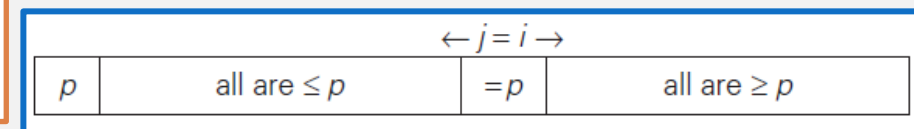
If $i < j$, we exchange $A[i]$ and $A[j]$ and resume the scans by incrementing i and decrementing j



If $i > j$, exchange the pivot with $A[j]$



If $i = j$, the value they are pointing to is equal to p . Thus, we have the subarray partitioned, with the split position $s = i = j$



Divide & Conquer

Sorting: Quicksort

Quicksort(A[0..7])

Partition(A[0..7]) ??

i
5 3 1 9 8 2 4 7

i j
5 3 1 9 8 2 4 7

i j
5 3 1 4 8 2 9 7

i j
5 3 1 4 8 2 9 7

i j
5 3 1 4 2 8 9 7

j i
5 3 1 4 2 8 9 7

2 3 1 4 5 8 9 7

Partition(A[0..7]) => 4

Quicksort(A[0..3])

Partition(A[0..3]) ??

i j i j j i
2 3 1 4 2 3 1 4 2 1 3 4 2 1 3 4 1 2 3 4

Partition(A[0..3]) => 1

Quicksort(A[0..0])

1

Quicksort(A[2..3])

i=j
1 2 3 4 1 2 3 4

Quicksort(A[5..7])

Partition(A[5..7]) ??

i j i j j i
1 2 3 4 5 8 9 7 1 2 3 4 5 8 7 9 1 2 3 4 5 8 7 9

1 2 3 4 5 7 8 9

Partition(A[5..7]) => 6

Quicksort(A[5..5])

7

Quicksort(A[7..7])

9

Final result

1 2 3 4 5 7 8 9

Divide & Conquer

Sorting: Analysis of quicksort

- Time taken by QuickSort in general can be written as following

$$T(n) = T(s) + T(n-s-1) + \Theta(n)$$

Recursive calls

Partitioning

s is the number of elements which are smaller than pivot

Worst Case: The worst case occurs when the partition process always picks greatest or smallest element as pivot.

$$T(n) = T(0) + T(n-1) + \Theta(n) = T(n-1) + \Theta(n) \\ = \Theta(n^2)$$

Use the first element as pivot and the **input is pre-sorted**

Best Case: The best case occurs when the partition process always picks the middle element as pivot.

$$T(n) = 2 T(n/2) + \Theta(n) \\ = \Theta(n \log n)$$

Use the median of the array (Hard to find exact median)

Average Case:

$$C_{avg}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [(n+1) + C_{avg}(s) + C_{avg}(n-1-s)] \quad \text{for } n > 1, \\ C_{avg}(0) = 0, \quad C_{avg}(1) = 0.$$

random arrays — $\Theta(n \log n)$

$$C_{avg}(n) \approx 2n \ln n \approx 1.39n \log_2 n.$$

Divide & Conquer

Sorting: Analysis of quicksort

- Quicksort is usually faster than Mergesort or Heapsort on randomly ordered arrays of nontrivial sizes
- Some possible improvements
 - **Randomized quicksort**: selects a random element as pivot
 - **Median-of-three**: selects median of left-most, middle, and right-most elements as pivot
 - **Switching to insertion sort** on very small subarrays, or not sorting small subarrays at all and finish the algorithm with insertion sort applied to the entire nearly sorted array
 - **Modify partitioning**: three-way partition
- Quick Sort is preferred over MergeSort for sorting Arrays due to the use of extra $O(N)$ storage space.
- MergeSort is preferred over QuickSort for Linked Lists, we can insert items in the middle in $O(1)$ extra space and $O(1)$ time. MergeSort can be implemented without extra space.

Divide & Conquer

Binary Tree Traversal

- A **binary tree** T is defined as a finite set of nodes that is either empty or consists of a root and two disjoint binary trees T_L and T_R called, respectively, the left and right subtree of the root.
- The most important divide-and-conquer algorithms for binary trees :
 - **preorder traversal**: root-left-right
 - **inorder traversal** : left-root-right
 - **postorder traversal**: left-right-root
 - **Height**: Length of the longest path from the root to a leaf

ALGORITHM $Height(T)$

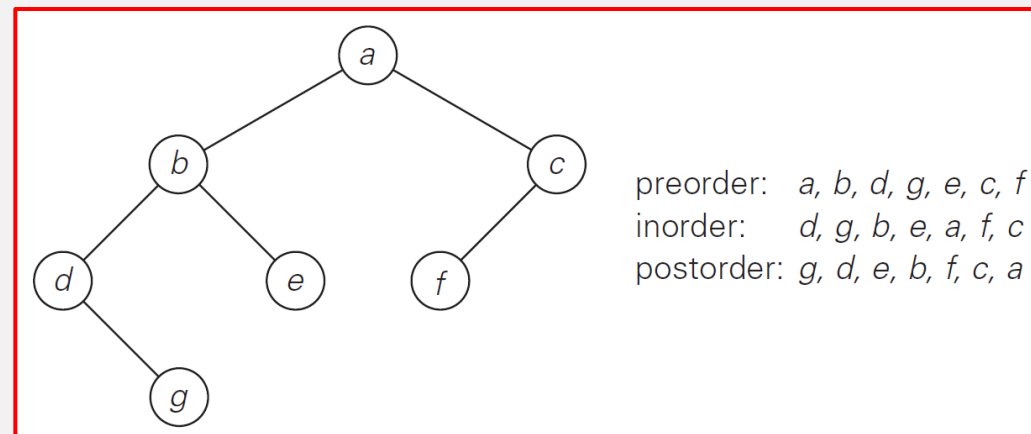
//Computes recursively the height of a binary tree

//Input: A binary tree T

//Output: The height of T

if $T = \emptyset$ **return** -1

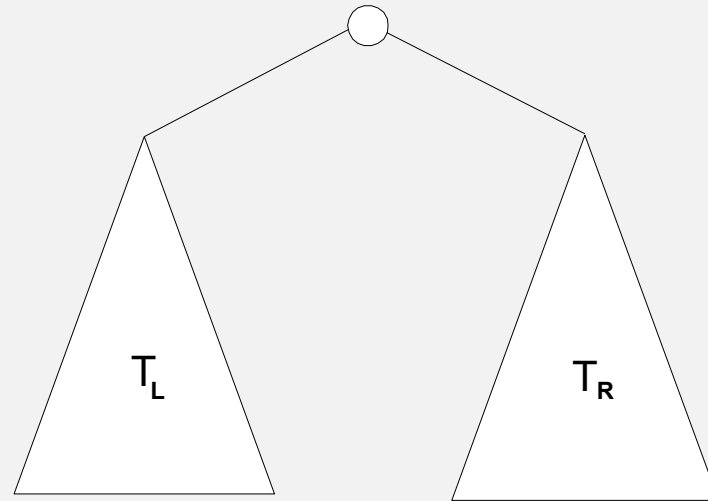
else return $\max\{Height(T_{left}), Height(T_{right})\} + 1$



Divide & Conquer

Binary Tree Traversal

Ex. 2: Computing the height of a binary tree



- $h(T) = \max\{h(T_L), h(T_R)\} + 1$ if $T \neq \emptyset$ and $h(\emptyset) = -1$
- Efficiency: $\Theta(n)$

Divide & Conquer

The Closest-Pair Problem

ALGORITHM *EfficientClosestPair*(P, Q)

if $n \leq 3$

return the minimal distance found by the brute-force algorithm

else

copy the first $\lfloor n/2 \rfloor$ points of P to array P_L

copy the same $\lfloor n/2 \rfloor$ points from Q to array Q_L

copy the remaining $\lfloor n/2 \rfloor$ points of P to array P_R

copy the same $\lfloor n/2 \rfloor$ points from Q to array Q_R

$d_L \leftarrow \text{EfficientClosestPair}(P_L, Q_L)$

$d_R \leftarrow \text{EfficientClosestPair}(P_R, Q_R)$

$d \leftarrow \min\{d_L, d_R\}$

$m \leftarrow P[\lfloor n/2 \rfloor - 1].x$

copy all the points of Q for which $|x - m| < d$ into array $S[0..num - 1]$

$d_{\min sq} \leftarrow d^2$

for $i \leftarrow 0$ to $num - 2$ do

$k \leftarrow i + 1$

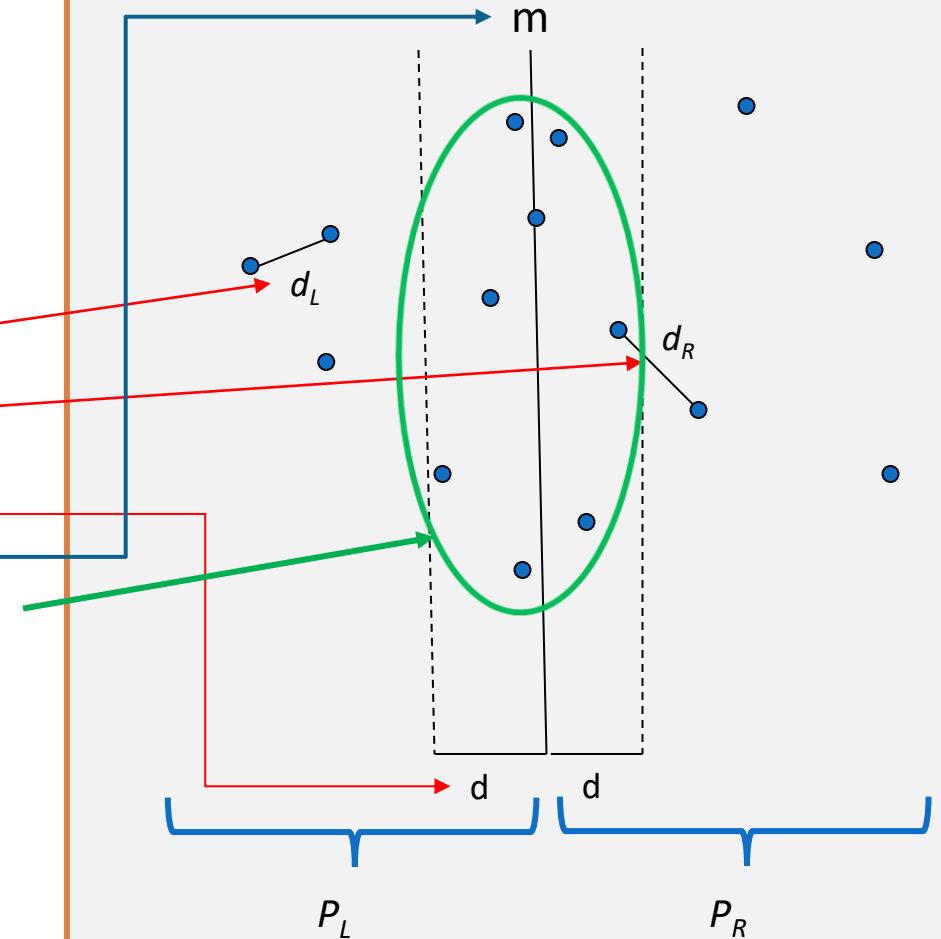
while $k \leq num - 1$ and $(S[k].y - S[i].y)^2 < d_{\min sq}$

$d_{\min sq} \leftarrow \min((S[k].x - S[i].x)^2 + (S[k].y - S[i].y)^2, d_{\min sq})$

$k \leftarrow k + 1$

return $\text{sqrt}(d_{\min sq})$

$$T(n) = 2T(n/2) + \Theta(n) \in (n \log n)$$



Divide & Conquer

The Maximum Subsequence Sum Problem

- Split the problem into two roughly equal sub-problems, which are then solved recursively
- Patch together the two solutions of the sub-problems to arrive at a solution for the whole problem
- The maximum subsequence sum can be:
 1. Entirely in the left half of the input,
 2. Entirely in the right half of the input,
 3. It crosses the middle and is in both halves.

First half				Second half			
4	-3	5	-2	-1	2	6	-2

Divide & Conquer

The Maximum Subsequence Sum Problem

- The first two cases can be solved recursively.
- For the last case:
 - find the largest sum in the first half that includes the last element in the first half,
 - find the largest sum in the second half that includes the first element in the second half, and
 - add these two sums together.

The Maximum Subsequence Sum Problem



Example: 8 numbers in a sequence,

4 -3 5 -2 -1 2 6 -2

- Max subsequence sum for first half = 6 ("4, -3, 5")
- Max subsequence sum for second half = 8 ("2, 6")
- Max subsequence sum for first half ending at the last element is 4 ("4, -3, 5, -2")
- Max subsequence sum for second half starting at the first element is 7 ("-1, 2, 6")
- Max subsequence sum spanning the middle is 11
("4, -3, 5, -2, -1, 2, 6")
- Max subsequence spans the middle
"4, -3, 5, -2, -1, 2, 6"

Divide & Conquer

The Maximum Subsequence Sum Problem

- Input: A, Left, Right
- Output: MaxSubSum of A
- MaxSubSum(A, Left, Right)

If Left == Right return A[Left] $O(1)$

Else

- Find MaxSubSum(A, Left, (Left + Right) / 2) $T(N/2)$
- Find MaxSubSum(A, (Left + Right) / 2 + 1, Right) $T(N/2)$
- Find MaxSubSum that contains both A[(Left + Right)/2] and A[(Left + Right) / 2 + 1] $O(N)$
- Return the maximum for the three sequences found $O(1)$

$$T(N) = 2T(N/2) + N = O(N \log N)$$

Try it